
Instructor notes on backpropagation for Spiking Neural Networks via Surrogate Gradient

Riccardo Mazzieri

Department of Information Engineering
University of Padova
riccardo.mazzieri@phd.unipd.it

Yesmine Abdennadher

Department of Information Engineering
University of Padova
yesmine.abdennadher@unipd.it

Michele Rossi

Department of Information Engineering
Department of Mathematics “Tullio Levi-Civita”
University of Padova
michele.rossi@unipd.it

Abstract

In these notes, we discuss backpropagation on Spiking Neural Networks (SNNs) via surrogate gradient descent, providing explicit mathematical derivations.

1 Problem definition

Biological neurons operate based on short temporal signals, called spikes, which are generated in response to external stimuli. The spikes are almost identical to one another, so the information that they carry *cannot be related to their shape*. On the other hand, spikes travel asynchronously and their temporal patterns is the key to how information processing happens in the human brain. We are currently able to track the input/output response of single neurons and of complex networks rather accurately. A popular mathematical model to do so is the so called Leaky Integrate and Fire (LIF) neuron model. Its popularity is due to the fact that it strikes a good balance between accuracy in the description of the biological processes in the brain and computational tractability. For an accurate description of how spiking networks operate from a biological perspective, the reader is referred to Gerstner et al. (2014). In what follow, we assume that the reader is familiar with the LIF model, considering its discrete time version, we first assemble a network of LIF neurons and then devise a procedure to train its weights.

We denote by $u_i^{(\ell)}[t]$ the membrane potential of the i -th neuron of layer ℓ at time index t , by $I_i^{(\ell)}[t]$ the input current to the i -th neuron of layer ℓ at time index t , and by $S_i^{(\ell)}[t]$ the spike output of the i -th neuron of layer ℓ at time index t , as a function of the membrane potential. This can be seen as an activation function. Finally, we denote by $W_{ij}^{(\ell)}$ the entries of the matrix of synaptic weights connecting layer $\ell - 1$ to layer ℓ .

This is the set of equations for a SNN based on LIF (Leaky Integrate and Fire) neurons.

$$S_i^{(\ell)}[t] = \Theta(u_i^{(\ell)}[t] - \theta), \quad (1)$$

$$u_i^{(\ell)}[t] = \beta u_i^{(\ell)}[t-1] + I_i^{(\ell)}[t] - \theta S_i^{(\ell)}[t-1], \quad (2)$$

$$I_i^{(\ell)}[t] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[t], \quad (3)$$

where $\Theta(\cdot)$ is the Heaviside step function, which models the non linear response of the spiking neuron. In particular, the neuron “spikes” (namely, generates a short pulse) when its membrane potential $u_i[t]$ exceeds a threshold θ . Hence, signal $S_i^{(\ell)}[t]$ is the input spike train to layer ℓ . After spiking, a so called “reset” mechanism is accounted for. In a biological neuron this corresponds to the discharge of the membrane potential after outputting a spike signal. In the above equation, this reset mechanism is implemented by removing $\theta S_i^{(\ell)}[t - 1]$ to the membrane potential.

A key problem with (1) is that the Heaviside function is not differentiable in $x = \theta$. This means that backpropagation cannot be natively used with this definition. One possible solution to this is to resort to the surrogate gradient approach. This method consists in keeping the Heaviside function as it is during the forward pass, but swapping its derivative with the derivative of a smooth differentiable approximation, which we denote by $\tilde{S}(u)$. There exist several choices for $\tilde{S}(u)$; some of them are listed in Table 1.

Table 1: Different choices of surrogate functions and their derivatives.

Name	$\tilde{S}(u)$	$\partial \tilde{S} / \partial u$
Arctan	$\frac{1}{\pi} \arctan(\pi u)$	$\frac{1}{1+(\pi u)^2}$
Sigmoid	$\frac{1}{1+\exp(-u)}$	$\tilde{S}(u)(1 - \tilde{S}(u))$
Fast sigmoid	$\frac{u}{1+ u }$	$\frac{1}{(1+ u)^2}$
Dspike	$\begin{cases} 0 & u < 0 \\ \frac{\tanh(b(u-0.5)) + \tanh(b/2)}{2 \tanh(b/2)} & 0 \leq u \leq 1 \\ 1 & u > 1 \end{cases}$	$\begin{cases} 0 & \text{if } u < 0 \\ \frac{b \operatorname{sech}^2(b(u-0.5))}{2 \tanh(b/2)} & \text{if } 0 \leq u \leq 1 \\ 0 & \text{if } u > 1 \end{cases}$

In the following, we consider a small network of LIF neurons following the dynamics described in (1)-(3), consisting of one single hidden layer with H neurons and an output layer. The input to the network consists of a sequence of T N -dimensional feature vectors $\mathbf{X}[t]$, $t = 1, \dots, T$, where we make no assumption on the specific encoding scheme of the input (i.e., the input can or cannot be in spike form). For this example, we will tackle a classification problem with a total of K classes (which will be the number of output neurons). Furthermore, we choose to focus directly on the membrane potential of the last layer as an output decoding strategy, instead of employing additional spike-decoding schemes.

The exact definition of the output at time t , $\mathbf{o}[t] = (o_1[t], \dots, o_K[t])^T$, depends on the values taken by the membrane potential $u_i[t]$, $i = 1, \dots, K$ of each neuron i in the last layer from the first timestep (time 1) up to and including to timestep t . The behavior of the output layer, often called *readout* layer, may differ from that of inner layers.

Two popular approaches are typically pursued:

- the output $o_i[t]$ for output neuron $i = 1, \dots, K$ at timestep t is computed as the maximum of $u_i[t]$ across timesteps $1, 2, \dots, t$, i.e.: $o_i[t] = \max_{t'=1, \dots, t} u_i[t']$.
- the output $o_i[t]$ is obtained by integrating the membrane potentials for the same neuron across timesteps $1, 2, \dots, t$, i.e., $o_i[t] = \sum_{t'=1}^t u_i[t']$.

Note that other ways of implementing the readout layer are possible, as discussed in Eshraghian et al. (2023).

In these notes, we choose the cumulative sum. After evaluating the output membrane potentials $\mathbf{o}[t]$, the outputs are then run through the usual `softmax` function to get a probability distribution of values, which constitute the final network prediction $\hat{y}_i[t]$. More rigorously, these two quantities are defined as follows:

$$\hat{y}_i[t] = \text{softmax}(\mathbf{o}[t])_i = \frac{\exp(o_i[t])}{\sum_{k=1}^K \exp(o_k[t])}, \quad i = 1, \dots, K, \quad (4)$$

$$o_i[t] = \sum_{j=1}^t u_i^{(2)}[j], \quad (5)$$

where the apex ⁽²⁾ refers to the membrane potentials of the neurons in the second (output) layer. Note that in (5), by only considering time indexes smaller than or equal to t , we are ensuring causality. We can now define the loss at timestep t , which is the usual cross entropy loss function:

$$\mathcal{L}[t] = - \sum_{k=1}^K y_k \log(\hat{y}_k[t]), \quad (6)$$

where $\mathbf{y} = (y_1, \dots, y_K)^T$ is the one-hot encoded *label* vector.

We now have all the necessary means to compute a loss value for each timestep $t = 1, \dots, T$. However, it will suffice to just optimize the loss value at the last timestep T , as, for how we choose to define it, it already accounts for all previous values of the membrane potential. Therefore, the final loss term is defined as:

$$\mathcal{L} = \mathcal{L}[T] = - \sum_{k=1}^K y_k \log(\hat{y}_k[T]). \quad (7)$$

Before going through the computations, we write all the necessary equations in a compact vector form, and instead of using a generic layer index ℓ , we will use the explicit layer indices. Note that, for the output layer we ignore the spike activity. Hence, for the output neurons we remove the reset term and allow their membrane potential to grow indefinitely large. Let $\mathbf{W}^{(1)} \in \mathbb{R}^{H \times N}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{K \times H}$ be the weights matrices of layer 1 and layer 2, respectively.

The full system of equations describing the considered two-layer neural network is:

$$\mathbf{I}^{(1)}[t] = \mathbf{W}^{(1)} \mathbf{X}[t] \in \mathbb{R}^H \quad (8)$$

$$\mathbf{u}^{(1)}[t] = \beta \mathbf{u}^{(1)}[t-1] + \mathbf{I}^{(1)}[t] - \overbrace{\theta \mathbf{S}^{(1)}[t-1]}^* \in \mathbb{R}^H \quad (9)$$

$$\mathbf{S}^{(1)}[t] = \Theta(\mathbf{u}^{(1)}[t] - \theta) \in \mathbb{R}^H \quad (10)$$

$$\mathbf{I}^{(2)}[t] = \mathbf{W}^{(2)} \mathbf{S}^{(1)}[t] \in \mathbb{R}^K \quad (11)$$

$$\mathbf{u}^{(2)}[t] = \beta \mathbf{u}^{(2)}[t-1] + \mathbf{I}^{(2)}[t] \in \mathbb{R}^K \quad (12)$$

$$\mathbf{o}[t] = \sum_{j \leq t} \mathbf{u}^{(2)}[j] \in \mathbb{R}^K \quad (13)$$

$$\hat{\mathbf{y}}[t] = \text{softmax}(\mathbf{o}[t]) \in \mathbb{R}^K \quad (14)$$

$$\mathcal{L} = \mathcal{L}[T] = -\langle \mathbf{y}, \log(\hat{\mathbf{y}}[T]) \rangle. \quad (15)$$

$$(16)$$

Where the starred reset term in (9) will be removed in the backward pass, hence will not be considered Eshraghian (2024). The reason behind this choice is due to experimental results. From the iterated application of backpropagation with a surrogate function, over different datasets and network models, superior performance was consistently observed by removing the reset term during the backward pass. From a theoretical standpoint, the reset term should be included. A more in depth analysis is required to fully understand the mathematical model underpinning this surrogate gradient approach.

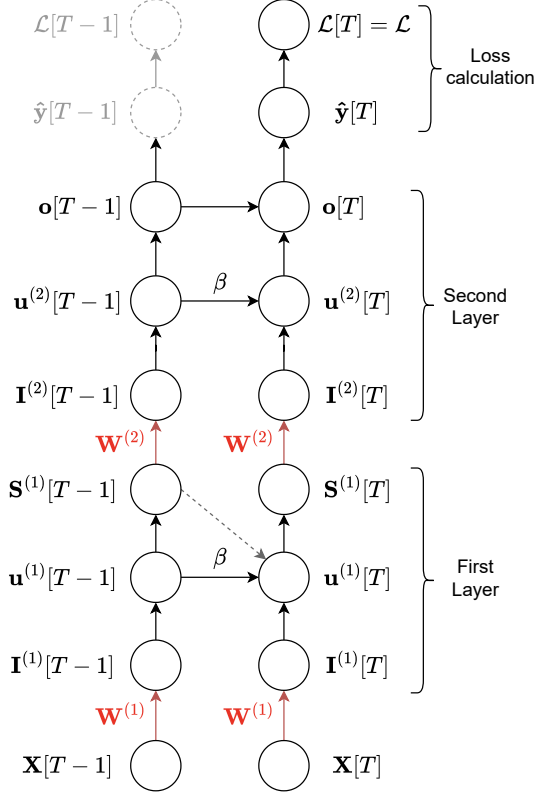


Figure 1: Diagram of the necessary computations, to use as reference for the backward pass. The gray dashed arrow depicts the reset mechanism of the first layer, which is present in the forward pass, but will be ignored in the backward pass.

Our final objective will be to compute the derivative of the final loss function $\mathcal{L}$ with respect to the two weight matrices, i.e., we will need to compute $\partial\mathcal{L}/\partial\mathbf{W}^{(1)}$ and $\partial\mathcal{L}/\partial\mathbf{W}^{(2)}$.

In Fig. 1 we show a diagram of the computational graph for the considered neural network.

2 Gradient computation

2.1 Computation of $\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(2)}}$

We compute the derivatives entry-wise, i.e., with respect to each entry of $\mathbf{W}^{(2)}$ and then re-write them in matrix form. By applying the chain rule of derivatives we can write:

$$\frac{\partial\mathcal{L}}{\partial W_{ij}^{(2)}} = \frac{\partial\mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(2)}} = \frac{\partial\mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \frac{\partial u_i^{(2)}[t]}{\partial W_{ij}^{(2)}}. \quad (17)$$

The first equality only considers the derivative of the loss with respect to the i -th component of the output, since that is the only component affected by the weight $W_{ij}^{(2)}$ (this is easy to verify by applying the definition of matrix-vector multiplication).

The second equality simply substitutes $o_i[T]$ using equation (5) and uses the linearity of the derivative. We now compute the first term, the derivative of the loss with respect to the output, as follows.

$$\frac{\partial \mathcal{L}}{\partial o_i[T]} = - \sum_{k=1}^K y_k \frac{\partial \log \hat{y}_k[T]}{\partial o_i[T]} = - \sum_{k=1}^K y_k \frac{1}{\hat{y}_k[T]} \frac{\partial \hat{y}_k[T]}{\partial o_i[T]}. \quad (18)$$

From (4) we obtain:

$$\frac{\partial \hat{y}_k[T]}{\partial o_i[T]} = \begin{cases} \hat{y}_i[T] (1 - \hat{y}_i[T]) & k = i \\ -\hat{y}_i[T] \hat{y}_k[T] & k \neq i. \end{cases} \quad (19)$$

Now using 19 back into 18, we obtain:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial o_i[T]} &= \underbrace{-y_i (1 - \hat{y}_i[T])}_{k=i} + \sum_{k \neq i} y_k \hat{y}_i[T] = -y_i + y_i \hat{y}_i[T] + \sum_{k \neq i} y_k \hat{y}_i[T] = \\ &= -y_i + \hat{y}_i[T] \underbrace{\sum_{k=1}^K y_k}_1 = \hat{y}_i[T] - y_i. \end{aligned} \quad (20)$$

In vector notation we have

$$\frac{\partial \mathcal{L}}{\partial \mathbf{o}[T]} = \hat{\mathbf{y}}[T] - \mathbf{y} \quad (21)$$

For the second term, $\frac{\partial u_i[t]}{\partial W_{ij}^{(2)}}$, for any $t = 1, \dots, T$. By iterated use of equation (12), we have:

$$\frac{\partial u_i^{(2)}[t]}{\partial W_{ij}^{(2)}} = \beta \frac{\partial u_i^{(2)}[t-1]}{\partial W_{ij}^{(2)}} + \frac{\partial I_i^{(2)}[t]}{\partial W_{ij}^{(2)}} \quad (22)$$

$$= \beta^2 \frac{\partial u_i^{(2)}[t-2]}{\partial W_{ij}^{(2)}} + \beta \frac{\partial I_i^{(2)}[t-1]}{\partial W_{ij}^{(2)}} + \frac{\partial I_i^{(2)}[t]}{\partial W_{ij}^{(2)}} = \quad (23)$$

$$= \dots = \sum_{k=1}^t \beta^{t-k} \frac{\partial I_i^{(2)}[k]}{\partial W_{ij}^{(2)}}. \quad (24)$$

Where in the last equality we use the fact that $u_i^{(2)}[1] = I_i^{(2)}[1]$. Now, from equation (11) it immediately follows that:

$$\frac{\partial I_i^{(2)}[k]}{\partial W_{ij}^{(2)}} = S_j^{(1)}[k]. \quad (25)$$

By substituting the results in equation (17) we have:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(2)}} = (\hat{y}_i[T] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} S_j^{(1)}[k]. \quad (26)$$

To express this derivative in vector notation, we can define the lower triangular matrix $\mathbf{B} \in \mathbb{R}^{T \times T}$ as follows:

$$\mathbf{B}_{ij} := \begin{cases} \beta^{i-j} & \text{if } i \geq j \\ 0 & \text{otherwise.} \end{cases} \quad (27)$$

We also denote with $\mathbf{S}^{(1)}$ the $\mathbb{R}^{T \times H}$ matrix of the spike activity of the first layer, and with $\mathbf{1}$ the H -dimensional (column) vector of all ones. With these definitions, equation (26) can be written in vector form as follows:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = (\hat{\mathbf{y}}[T] - \mathbf{y}) \cdot \left(\mathbf{1}^T \mathbf{B} \mathbf{S}^{(1)} \right) \quad (28)$$

This last product can be seen as a matrix product between a $K \times 1$ and a $1 \times H$ matrix, so the result yields a $K \times H$ matrix, as expected.

2.2 Computation of $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$

Applying the same logic as above, we can write:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(1)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \frac{\partial u_i[t]}{\partial W_{ij}^{(1)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{\partial I_i^{(2)}[k]}{\partial W_{ij}^{(1)}}. \quad (29)$$

$$(30)$$

which simply follows from the linearity of the derivative. Now, we delve into the computation of $\frac{\partial I_i^{(2)}[k]}{\partial W_{ij}^{(1)}}$.

$$\frac{\partial I_i^{(2)}[k]}{\partial W_{ij}^{(1)}} = \sum_{z=1}^H W_{iz}^{(2)} \frac{\partial S_z^{(1)}[k]}{\partial W_{ij}^{(1)}} = W_{ii}^{(2)} \frac{\partial S_i^{(1)}[k]}{\partial W_{ij}^{(1)}} \quad (31)$$

where the first equality just applies equation (11), and the second equality follows from the fact that $\frac{\partial S_z^{(1)}[k]}{\partial W_{ij}^{(1)}} = 0$ for all $z \neq i$. To proceed, we therefore need to compute $\frac{\partial S_i^{(1)}[k]}{\partial W_{ij}^{(1)}}$. By applying the definition of the spike function, the chain rule of derivatives, and recalling that we have to swap it with the *surrogate spike function*, we can write:

$$\frac{\partial S_i^{(1)}[k]}{\partial W_{ij}^{(1)}} = \frac{\partial \tilde{S}_i^{(1)}[k]}{\partial u_i^{(1)}[k]} \frac{\partial u_i^{(1)}[k]}{\partial W_{ij}^{(1)}} = \frac{1}{1 + (\pi u_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} \frac{\partial I_i^{(1)}[s]}{\partial W_{ij}^{(1)}} \quad (32)$$

where in the second equality we apply the arctan surrogate gradient, and decompose the derivative of $u_i^{(1)}[t]$ using the same reasoning applied to equation (22). Finally, by substituting all the intermediate results we have just computed in equation (30), we can write the final derivative:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(1)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{W_{ii}^{(2)}}{1 + (\pi u_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} X_j[s] = \quad (33)$$

$$= (\hat{y}_i[T] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{W_{ii}^{(2)}}{1 + (\pi u_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} X_j[s]. \quad (34)$$

where the last equality follows from the fact that $\frac{\partial I_i^{(1)}[s]}{\partial W_{ij}^{(1)}} = X_j[s]$.

Remark no. 1: the parameters θ (membrane threshold) and β (membrane decaying factor) can be also learned with the backpropagation mechanism. In this case, it is sufficient to calculate the partial derivatives of the loss also with respect to these two weights and perform a gradient descent on these two parameters as well.

Remark no. 2: In standard neural networks, for a given input vector $\mathbf{x}$ a neuron computes $f(\mathbf{W}\mathbf{x} + \mathbf{b})$, where $\mathbf{W}$ is a matrix of weights (to be learned), $\mathbf{b}$ is a vector of biases (to be also learned) and $f(\cdot)$ is a non-linearity (applied element-wise). The biases are key to ensuring that the neurons will act as classifiers with a decision boundary (a plane) that can be adjusted in terms of “angle” (using $\mathbf{W}$) and shifted away from the origin via the $\mathbf{b}$ parameter. With SNN, the non-linearity corresponds to the Heaviside function $H(\cdot)$ and the biases can be replaced by the parameter θ , which effectively shift the response of the non-linear classifier.

References

- Jason K. Eshraghian. 2024. Tutorial 5 - Training Spiking Neural Networks with SNNRToch. https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_5.html. Last accessed: December 2, 2024.
- Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, and Girish Dwivedi. 2023. Training Spiking Neural Networks Using Lessons From Deep Learning. *Proc. IEEE* (2023), 1016–1054.
- Wulfram Gerstner, Werner M. Kistler, Richard Naud, and Liam Paninski. 2014. *Neuronal Dynamics: from Single Neurons to Networks and Modes of Cognition*. Cambridge University Press.